

Implementação de Forward Propagation Configurável e Análise de Desempenho

Pedro Pierobon Marynowski
Departamento de Informática
Universidade Federal do Paraná – UFPR
Curitiba, Brasil
GRR20251729

Resumo—Este trabalho apresenta a implementação configurável do algoritmo de *Forward Propagation* para Redes Neurais Artificiais profundas. Foram desenvolvidas duas versões: uma vetorizada, utilizando álgebra linear otimizada, e uma não-vetorizada, baseada em laços de repetição. O estudo compara o desempenho computacional entre as abordagens e analisa o impacto das funções de ativação Sigmoid e ReLU. Os resultados demonstram que a vetorização é indispensável para a escalabilidade de modelos de *Deep Learning*.

Index Terms—Redes Neurais, Vetorização, Forward Propagation, Desempenho, Python.

I. INTRODUÇÃO

O *Forward Propagation* é a etapa fundamental na inferência de redes neurais, onde os dados de entrada atravessam as camadas ocultas para gerar uma predição. Em arquiteturas profundas (*Deep Learning*), a eficiência dessa etapa é crítica.

O objetivo deste trabalho prático é implementar este algoritmo de forma configurável em relação aos hiperparâmetros de arquitetura (número de camadas e neurônios) e realizar testes de estresse para comparar implementações vetorizadas (usando a biblioteca NumPy) contra implementações ingênuas (laços *for* em Python).

II. DECISÕES DE PROJETO

A implementação foi realizada na linguagem Python (v3.x), escolhida por ser o padrão da indústria para Aprendizado de Máquina. Abaixo estão as principais decisões técnicas tomadas, baseadas no material da disciplina.

A. Bibliotecas e Estruturas de Dados

A biblioteca *NumPy* foi utilizada para a implementação vetorizada, permitindo operações matriciais eficientes. Para a implementação não-vetorizada, utilizou-se apenas listas nativas e a biblioteca *math* do Python, simulando um ambiente sem otimização de álgebra linear.

B. Convenção de Matrizes

Seguindo a convenção moderna de frameworks como TensorFlow e Keras (apresentada nos slides de “Implementação Avançada”), optou-se pela representação dos dados X com dimensão $(m, n_{features})$, onde m é o número de exemplos. Consequentemente, a matriz de pesos $W^{[l]}$ da camada l possui dimensão $(n^{[l-1]}, n^{[l]})$. A operação linear é definida como:

$$Z^{[l]} = A^{[l-1]} \cdot W^{[l]} + b^{[l]} \quad (1)$$

Isso difere da notação matemática clássica $(n^{[l]}, n^{[l-1]})$, mas facilita o *broadcasting* do vetor de bias b e a manipulação de *batches* de dados.

C. Inicialização de Parâmetros

Os pesos W foram inicializados aleatoriamente a partir de uma distribuição normal padrão, multiplicados por um fator de escala de 0.01 para manter os valores pequenos e evitar a saturação de funções como a Sigmoid nas primeiras iterações. Os biases b foram inicializados com zeros.

III. RESULTADOS DOS TESTES

Para validar a escalabilidade, foram definidos quatro cenários de teste com complexidade crescente.

A. Ambiente de Teste

Os testes foram executados no seguinte ambiente:

- **CPU:** Intel Core i7-11500H
- **RAM:** 16 GB DDR5
- **Dataset:** Matriz fixa de 1000 exemplos $\times$ 100 features.

B. Configuração das Redes

A Tabela I detalha as arquiteturas utilizadas nos testes de estresse.

Tabela I
CONFIGURAÇÕES DE REDE PARA TESTES DE ESTRESSE

Config.	Camadas Ocultas	Neurônios (Largura)
Pequena	2	[32, 16]
Média	4	[128, 64, 32, 16]
Grande	8	[128] (todas as camadas)
Extrema	10	[512] (todas as camadas)

C. Comparação de Desempenho

A Tabela II apresenta os tempos médios de execução para o *forward propagation*. Para as configurações “Grande” e “Extrema” na versão não-vetorizada, o tempo foi estimado executando-se uma fração reduzida do dataset, devido à inviabilidade temporal de processar o conjunto completo.

D. Análise Gráfica

A Figura 1 ilustra a comparação entre as implementações utilizando uma escala logarítmica.

Tabela II
TEMPO MÉDIO DE EXECUÇÃO (SEGUNDOS)

Config.	Implementação	Ativação	Tempo (s)
Média	Não-Vetorizada	ReLU	5.1432
	Vetorizada	Sigmoide	0.0028
	Vetorizada	ReLU	0.0017
Grande	Não-Vetorizada	ReLU	27.6580
	Vetorizada	ReLU	0.0064
Extrema	Não-Vetorizada	ReLU	≈ 532.27
	Vetorizada	ReLU	0.0561

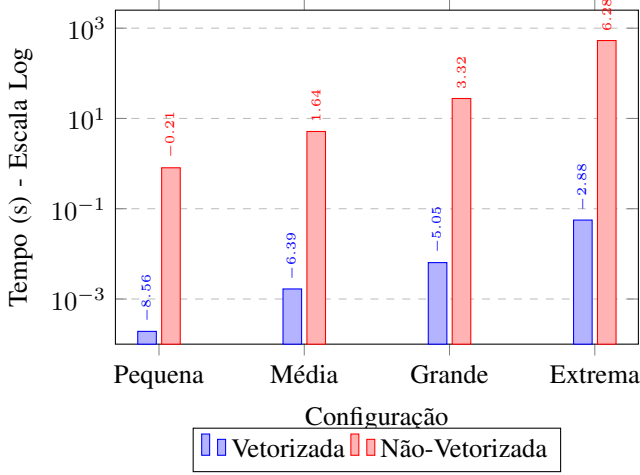


Figura 1. Comparação de desempenho (Barras em Escala Logarítmica).

IV. ANÁLISE E CONCLUSÕES

A análise dos resultados revela uma disparidade de magnitude no desempenho.

A. Impacto da Vetorização

A implementação vetorizada mostrou-se ordens de magnitude mais rápida. Na configuração “Extrema”, enquanto a versão vetorizada levou apenas **0.056 segundos**, a versão com laços levaria aproximadamente **532 segundos** (cerca de 9 minutos). Isso ocorre porque o NumPy delega as operações matriciais para bibliotecas de baixo nível (BLAS/LAPACK) escritas em C/Fortran, que utilizam instruções SIMD da CPU, enquanto os laços do Python sofrem com o *overhead* do interpretador.

B. Impacto da Função de Ativação

Observou-se que a função Rectified Linear Unit (ReLU) é computacionalmente mais eficiente que a Sigmoide. Na configuração “Extrema” vetorizada, a ReLU (0.0561s) foi cerca de 24% mais rápida que a Sigmoide (0.0745s). A Sigmoide envolve o cálculo de uma exponencial (e^{-z}), uma operação custosa, enquanto a ReLU envolve apenas uma comparação ($\max(0, z)$).

C. Conclusão

Conclui-se que a implementação não-vetorizada é inviável para redes neurais produtivas. A escalabilidade do *Deep Learning* depende estritamente do uso de álgebra linear otimizada. A implementação vetorizada demonstrou ser capaz de processar arquiteturas complexas em milissegundos, validando as decisões de projeto baseadas nos frameworks modernos como TensorFlow.